

Waterfall Model

Waterfall Model เป็นกระบวนการพัฒนาซอฟต์แวร์ที่เป็นลำดับขั้นตอน (Sequential Model) โดยการทำงานจะแบ่งออกเป็นเฟสที่ชัดเจนและทำทีละขั้นตอนแบบต่อเนื่องจากบนลงล่าง เหมือนกับน้ำตกที่ไหลลงจากระดับหนึ่งไปยังอีกระดับหนึ่ง ไม่สามารถย้อนกลับไปยังขั้นตอนก่อนหน้าได้

ขั้นตอนของ Waterfall Model

1. **Requirements Analysis (วิเคราะห์ความต้องการ)**
 - เก็บรวบรวมและวิเคราะห์ความต้องการของผู้ใช้งานหรือระบบ
 - จัดทำเอกสารความต้องการ (Requirements Specification)
2. **System Design (ออกแบบระบบ)**
 - กำหนดโครงสร้างของระบบ เช่น สถาปัตยกรรม การออกแบบฐานข้อมูล และอินเทอร์เฟซ
 - จัดทำเอกสารการออกแบบ (System Design Document)
3. **Implementation (การพัฒนา/เขียนโค้ด)**
 - แปลงการออกแบบระบบให้เป็นโค้ดโปรแกรมจริง
 - ทีมพัฒนาจะทำการเขียนโค้ดตามเอกสารที่ได้ออกแบบไว้
4. **Integration and Testing (รวมระบบและทดสอบ)**
 - รวมส่วนต่าง ๆ ของซอฟต์แวร์เข้าด้วยกัน
 - ทดสอบระบบเพื่อหาข้อผิดพลาดหรือบั๊ก (Bugs)
5. **Deployment (ติดตั้งระบบ)**
 - ส่งมอบและติดตั้งซอฟต์แวร์ให้ผู้ใช้งานหรือระบบจริง
6. **Maintenance (การบำรุงรักษา)**
 - แก้ไขปัญหาที่เกิดขึ้นในระบบจริง
 - ปรับปรุงระบบตามความต้องการใหม่ ๆ

ข้อดีของ Waterfall Model

- ง่ายต่อการวางแผนและการบริหารจัดการ
- มีเอกสารที่ครบถ้วนในแต่ละขั้นตอน
- เหมาะสำหรับโครงการที่มีความต้องการชัดเจนตั้งแต่ต้น

ข้อเสียของ Waterfall Model

- ยืดหยุ่นน้อย หากความต้องการเปลี่ยนแปลง
- ไม่เหมาะกับโครงการที่มีความซับซ้อนสูงหรือมีการเปลี่ยนแปลงบ่อย
- ปัญหาหรือข้อผิดพลาดในขั้นตอนแรกอาจมีผลกระทบต่อขั้นตอนถัดไป

Waterfall Model มักใช้ในโครงการที่มีความต้องการและเป้าหมายที่ชัดเจน เช่น ระบบที่ต้องการความเสถียรสูง หรือการพัฒนาซอฟต์แวร์สำหรับองค์กรที่มีขั้นตอนการอนุมัติที่เข้มงวด

Agile Model

Agile Model เป็นกระบวนการพัฒนาซอฟต์แวร์แบบยืดหยุ่น (Flexible Model) ซึ่งเน้นการแบ่งการทำงานออกเป็นช่วงสั้นๆ ที่เรียกว่า Iterative หรือ Sprint โดยในแต่ละช่วงจะมีการวางแผน พัฒนา ทดสอบ และส่งมอบซอฟต์แวร์ที่ใช้งานได้จริงเป็นส่วนย่อย และปรับปรุงตามความต้องการของผู้ใช้งานในแต่ละรอบ

ขั้นตอนของ Agile Model

1. **Concept and Planning (แนวคิดและการวางแผน)**
 - วิเคราะห์ความต้องการในเบื้องต้น
 - วางแผนระยะสั้นสำหรับ Sprint แรก
2. **Sprint Execution (การดำเนินงานในแต่ละช่วง)**
 - แบ่งการพัฒนาเป็น Sprint โดยแต่ละ Sprint ใช้เวลา 2-4 สัปดาห์
 - ทีมงานจะทำงานร่วมกันเพื่อพัฒนาและทดสอบซอฟต์แวร์ในแต่ละรอบ
3. **Testing and Feedback (การทดสอบและรับฟังความคิดเห็น)**
 - ทดสอบฟังก์ชันที่พัฒนาขึ้นในแต่ละ Sprint
 - รับฟังความคิดเห็นจากผู้ใช้งานเพื่อนำไปปรับปรุงใน Sprint ถัดไป
4. **Delivery (การส่งมอบ)**
 - ส่งมอบซอฟต์แวร์ที่ใช้งานได้จริงเป็นส่วนย่อยในแต่ละ Sprint
 - ปรับปรุงและพัฒนาอย่างต่อเนื่องจนเสร็จสมบูรณ์

ข้อดีของ Agile Model

- ยืดหยุ่นสูง: สามารถปรับเปลี่ยนตามความต้องการของผู้ใช้งานได้ตลอดเวลา
- ส่งมอบเร็ว: ซอฟต์แวร์ที่ใช้งานได้จริงสามารถส่งมอบในแต่ละ Iteration
- การทำงานร่วมกัน: ทีมงานและผู้ใช้งานมีการสื่อสารอย่างใกล้ชิด

ข้อเสียของ Agile Model

- การบริหารจัดการยาก: ต้องการการทำงานแบบมีประสิทธิภาพสูงและความร่วมมือจากทุกฝ่าย
- ขาดเอกสารที่ครบถ้วน: เอกสารอาจไม่สมบูรณ์เมื่อเทียบกับ Waterfall Model
- อาจไม่เหมาะกับทุกโครงการ: โดยเฉพาะโครงการที่มีขอบเขตและเป้าหมายชัดเจนแต่ต้น

Agile Model เหมาะสำหรับโครงการที่ต้องการความยืดหยุ่น เช่น

- การพัฒนาแอปพลิเคชันมือถือ
- ซอฟต์แวร์ที่เปลี่ยนแปลงตามความต้องการของผู้ใช้งานอย่างรวดเร็ว
- โครงการที่เน้นการมีส่วนร่วมของผู้ใช้งานตลอดกระบวนการพัฒนา

Spiral Model คืออะไร

Spiral Model เป็นโมเดล SDLC ที่นำจุดแข็งของ Waterfall Model มาพัฒนาให้ดีขึ้น เพราะแต่เดิมหากขั้นตอนไหนผิดพลาดจะไม่สามารถแก้ไขได้ ต้องรอจบกระบวนการทั้งหมดก่อนถึงจะแก้ไขได้ตั้งแต่ขั้นแรกและเรียงลำดับขั้นไปเรื่อย ๆ ทำให้มีความเสี่ยงที่การพัฒนาซอฟต์แวร์จะไม่สำเร็จ

จึงมีการปรับเปลี่ยนรูปแบบเป็น Spiral model หรือโมเดลรูปเกลียว ที่เน้นการประเมินความเสี่ยงของโปรเจกต์ ด้วยการแบ่งโครงสร้างออกเป็นโปรเจกต์กลุ่มย่อย ช่วยให้การปรับเปลี่ยน requirement ในระหว่างพัฒนาทำได้ง่ายขึ้น ตลอดจนประเมินความเสี่ยงกับความต่อเนื่องของโปรเจกต์ในเรื่องค่าใช้จ่าย SDLC model นี้เหมาะกับโปรเจกต์ที่มีความซับซ้อนสูง และมีความเสี่ยงที่ต้องควบคุมอย่างเข้มงวด

ขั้นตอนของ Spiral Model

1. **Planning (การวางแผน)**
 - ระบุวัตถุประสงค์ของโครงการ ความต้องการของผู้ใช้งาน และทรัพยากรที่จำเป็น
 - วางแผนรอบ (Spiral) แรกและกำหนดขอบเขตงาน
2. **Risk Analysis (การวิเคราะห์ความเสี่ยง)**
 - วิเคราะห์ความเสี่ยงที่อาจเกิดขึ้น เช่น ความเสี่ยงด้านเทคนิค การเงิน และเวลา
 - หากพบความเสี่ยงที่สำคัญ ให้ปรับกลยุทธ์เพื่อลดความเสี่ยง
3. **Engineering (การพัฒนา)**
 - ออกแบบ พัฒนา และสร้างต้นแบบ (Prototype)
 - ทดสอบระบบในแต่ละรอบเพื่อปรับปรุงให้ตรงตามความต้องการ
4. **Evaluation (การประเมินผล)**
 - ตรวจสอบผลลัพธ์กับผู้ใช้งานเพื่อให้ได้ข้อเสนอแนะ
 - นำข้อเสนอแนะไปปรับปรุงในรอบถัดไป

ข้อดีของ Spiral Model

- ยืดหยุ่นและปรับปรุงได้ต่อเนื่อง: สามารถเปลี่ยนแปลงความต้องการหรือแก้ไขปัญหาในแต่ละรอบได้
- เน้นการวิเคราะห์ความเสี่ยง: ลดโอกาสเกิดปัญหาใหญ่ในภายหลัง
- เหมาะสำหรับโครงการขนาดใหญ่: โดยเฉพาะโครงการที่ซับซ้อนหรือมีความไม่แน่นอน

ข้อเสียของ Spiral Model

- ต้นทุนสูง: กระบวนการวิเคราะห์ความเสี่ยงและการวนซ้ำอาจใช้เวลาและทรัพยากรจำนวนมาก
- ซับซ้อน: ต้องการทีมงานที่มีความเชี่ยวชาญสูง
- ไม่เหมาะกับโครงการขนาดเล็ก: เนื่องจากการวนซ้ำแต่ละครั้งอาจเกินความจำเป็น

Spiral Model เหมาะสำหรับโครงการที่มีความเสี่ยงสูงและมีความซับซ้อน เช่น

- ระบบที่ต้องการความปลอดภัยสูง
- โครงการที่ยังไม่ชัดเจนในความต้องการทั้งหมด
- การพัฒนาาระบบที่ต้องการปรับปรุงอย่างต่อเนื่อง

V-Shaped Model

V-Shaped model เป็นโมเดล SDLC ที่เน้นการทดสอบอย่างเข้มงวดในแต่ละขั้นตอน มีลำดับขั้นตอนที่ชัดเจน สามารถย้อนกลับไปทำขั้นก่อนหน้าได้ โมเดลนี้เป็นการขยายและปรับปรุงมาจาก Waterfall model ด้วยการเพิ่มส่วนการทดสอบเข้ามาในทุกขั้นตอน มีจุดเด่นเรื่องการรับประกันว่าในแต่ละขั้นตอนการพัฒนาจะได้รับการทดสอบอย่างถี่ถ้วน ทำให้ข้อผิดพลาดหรือบั๊กถูกตรวจพบ และแก้ไขได้ตั้งแต่ระยะเริ่มต้นพัฒนาซอฟต์แวร์

ขั้นตอนของ V-Shaped Model

- Project and Requirements Planning : การจัดสรรทรัพยากร
- Product Requirements and Specification Analysis : กำหนด spec ที่สมบูรณ์ของ Software
- Architecture or High-Level Design : กำหนดวิธีการทำงานที่ตอบสนองต่อการออกแบบ Software
- Detailed Design : การพัฒนาอัลกอริทึมสำหรับองค์ประกอบต่าง ๆ
- Production, operation and maintenance : การเพิ่มประสิทธิภาพและการแก้ไข
- System and acceptance testing : ตรวจสอบสภาพแวดล้อมของ Software
- Integration and Testing : ตรวจสอบการเชื่อมต่อแต่ละ Module ว่าถูกต้องหรือไม่
- Unit testing : ตรวจสอบการทำงานของแต่ละ module ว่าทำงานถูกต้องตามเป้าหมาย
- Coding : เปลี่ยน Algorithm เป็น Software

ข้อดีของ V-Shaped model

- เน้นการวางแผนสำหรับการ Verification และ Validation ของผลิตภัณฑ์ในขั้นเริ่มต้นของการพัฒนาผลิตภัณฑ์
- งานที่ส่งมอบต้องสามารถทดสอบได้ทุกขั้นตอน
- สามารถติดตามความก้าวหน้าได้ทุกขั้นตอน
- เข้าใจง่ายและใช้งานได้ง่าย

ข้อเสียของ V-Shaped Model

- ยากต่อการจัดการเหตุการณ์ที่เกิดขึ้นพร้อมกัน
- ยากต่อการจัดการกับ Requirement ที่มีการเปลี่ยนแปลงบ่อยๆ
- ไม่มีระบบการจัดการความเสี่ยง (Risk Management)

V-Shaped Model เป็นโมเดลที่เน้นการตรวจสอบ(verification) และการรับรองความถูกต้อง (validation) ควบคู่กันไป อีกทั้งยังเป็นการทดสอบผลิตภัณฑ์ที่กระทำขนานกันไปกับการวางแผนในการพัฒนาซอฟต์แวร์ และสามารถติดตามความก้าวหน้าได้ทุกขั้นตอน

Iterative Model

Iterative Model เป็น แบบจำลองวงจรชีวิตแบบวนซ้ำไม่ได้เริ่มต้นด้วยการระบุข้อกำหนดทั้งหมด ในแบบจำลองนี้ การพัฒนาเริ่มต้นด้วยการระบุและนำซอฟต์แวร์เพียงบางส่วนไปใช้ จากนั้นจึงตรวจสอบเพื่อระบุข้อกำหนดเพิ่มเติม นอกจากนี้ ในแบบจำลองแบบวนซ้ำ กระบวนการวนซ้ำเริ่มต้นด้วยการนำข้อกำหนดซอฟต์แวร์ชุดเล็กๆ ไปใช้งานอย่างง่าย ๆ ซึ่งจะปรับปรุงเวอร์ชันที่พัฒนาขึ้นเรื่อยๆ อย่างต่อเนื่อง จนกระทั่งนำระบบทั้งหมดไปใช้งานและพร้อมที่จะนำไปใช้งาน แบบจำลองแบบวนซ้ำแต่ละรุ่นจะได้รับการพัฒนาในช่วงเวลาที่กำหนดและแน่นอน ซึ่งเรียกว่าการวนซ้ำ

ขั้นตอนของ Iterative Model

1. **ระยะการวางแผน:**นี่คือระยะแรกของแบบจำลองเชิงวนซ้ำ โดยที่ทีมงานจะมีการวางแผนอย่างเหมาะสม ซึ่งช่วยให้พวกเขาสามารถจัดทำแผนผังเอกสารข้อกำหนด กำหนดข้อกำหนดด้านซอฟต์แวร์หรือฮาร์ดแวร์ และเตรียมความพร้อมสำหรับขั้นตอนต่อไปของวงจรโดยทั่วไป
2. **ขั้นตอนการวิเคราะห์และการออกแบบ:**เมื่อการวางแผนสำหรับรอบการทำงานเสร็จสิ้นแล้ว การวิเคราะห์จะดำเนินการเพื่อระบุตรรกะทางธุรกิจที่เหมาะสม โมเดลฐานข้อมูล และเพื่อทราบข้อกำหนดอื่นๆ ของขั้นตอนนี้โดยเฉพาะ นอกจากนี้ ขั้นตอนการออกแบบยังเกิดขึ้นในขั้นตอนของโมเดลแบบวนซ้ำนี้ โดยที่ข้อกำหนดทางเทคนิคจะถูกกำหนดขึ้นเพื่อนำไปใช้เพื่อตอบสนองความต้องการของขั้นตอนการวิเคราะห์
3. **ระยะการนำไปปฏิบัติ:**นี่คือระยะที่สามและเป็นระยะที่สำคัญที่สุดของแบบจำลองเชิงวนซ้ำ ในขั้นตอนนี้ กระบวนการนำไปใช้จริงและการเข้ารหัสจะถูกดำเนินการ เอกสารการวางแผน ข้อกำหนด และการออกแบบทั้งหมดจนถึงจุดนี้จะถูกเข้ารหัสและนำไปใช้ในขั้นตอนการวนซ้ำเริ่มต้นของโครงการ
4. **ระยะการทดสอบ:**หลังจากที่ทำการเข้ารหัสและนำการวนซ้ำของการสร้างปัจจุบันไปใช้แล้ว จะเริ่มการทดสอบในรอบการทำงานเพื่อระบุและค้นหาจุดบกพร่องหรือปัญหาที่อาจเกิดขึ้นในซอฟต์แวร์
5. **ระยะประเมิน:**ระยะสุดท้ายของวงจรชีวิตแบบวนซ้ำคือระยะประเมิน ซึ่งทีมงานทั้งหมดพร้อมกับไคลเอนต์จะตรวจสอบสถานะของโครงการและตรวจสอบว่าเป็นไปตามข้อกำหนดที่แนะนำหรือไม่

ข้อดีของ Iterative Model

- ปรับเปลี่ยนและเพิ่มเติมฟีเจอร์ได้สะดวก เหมาะกับโปรเจกต์ซอฟต์แวร์ที่ requirement ไม่แน่นอน หรืออาจได้เปลี่ยนแปลงบ่อย
- ปัญหาข้อผิดพลาดถูกตรวจเจอได้เร็ว จากการทวนซ้ำขั้นตอนหลายรอบ
- มีการรับฟีดแบ็คจากผู้ใช้ระหว่างการพัฒนา และนำมาปรับปรุงซอฟต์แวร์ให้ตอบโจทย์ดีมากขึ้น

ข้อเสียของ Iterative Model

- งานเพิ่มจากการขยายขอบเขตไปเรื่อยๆแบบไม่มีสิ้นสุด (Scope Creep) การวนซ้ำแต่ละรอบจึงต้องอาศัยความระมัดระวังสูง

RAD Model

RAD model หรือ Rapid Application Development model คือ กระบวนการพัฒนาซอฟต์แวร์โดยอาศัยการสร้างต้นแบบโดยไม่มีการวางแผนเฉพาะใดๆ ในโมเดล RAD จะให้ความสำคัญกับการวางแผนน้อยลง และให้ความสำคัญกับงานการพัฒนามากขึ้น มุ่งเป้าไปที่การพัฒนาซอฟต์แวร์ในระยะเวลาอันสั้น

ขั้นตอนของ RAD Model

- 1.การสร้างแบบจำลองธุรกิจ : บนพื้นฐานการไหลเวียนของข้อมูลและการกระจายระหว่างช่องทางธุรกิจต่างๆ ผลิตภัณฑ์ได้รับการออกแบบ
- 2.การสร้างแบบจำลองข้อมูล : ข้อมูลที่รวบรวมจากการสร้างแบบจำลองธุรกิจจะถูกปรับแต่งเป็นชุดของออบเจกต์ข้อมูลที่มีความสำคัญต่อธุรกิจ
- 3.การสร้างแบบจำลองกระบวนการ : ออบเจกต์ข้อมูลที่ได้รับการประกาศในขั้นตอนการสร้างแบบจำลองข้อมูลจะได้รับการแปลงเพื่อให้เกิดการไหลของข้อมูลที่จำเป็นต่อการนำฟังก์ชันทางธุรกิจไปใช้
- 4.การสร้างแอปพลิเคชัน : เครื่องมืออัตโนมัติใช้สำหรับการสร้างซอฟต์แวร์ เพื่อแปลงกระบวนการและแบบจำลองข้อมูลให้เป็นต้นแบบ
- 5.การทดสอบและการหมุนเวียน : เนื่องจากต้นแบบได้รับการทดสอบแยกกันระหว่างการพัฒนาซ้ำทุกครั้ง เวลาการทดสอบโดยรวมจึงลดลงใน RAD

ข้อดีของ RAD Model

- มีความยืดหยุ่นและปรับตัวต่อการเปลี่ยนแปลงได้
- มีประโยชน์เมื่อต้องลดความเสี่ยงโดยรวมของโครงการ
- โมเดลนี้เน้นความเร็วการพัฒนาเป็นหลัก จึงส่งมอบซอฟต์แวร์ในเวลาอันสั้นได้

ข้อเสียของ RAD Model

- การลดขนาดระบบลง (Reduced Scalability) ทำให้มีความยืดหยุ่นน้อย
- เนื่องเพราะพัฒนามาจากระบบต้นแบบ

Big Bang Model

Big Bang model เป็นโมเดล SDLC ที่เน้นไปที่การเขียนโค้ดของซอฟต์แวร์โดยตรง ไม่เน้นการวางแผนหรือการทำงานที่ละเอียดอย่างเป็นระบบ การพัฒนาจะเริ่มต้นโดยไม่มีแผนที่ละเอียดและโครงสร้างไม่ชัดเจน เหมาะกับโปรเจกต์ขนาดเล็กที่มีนักพัฒนา 1-2 คน ทำงานร่วมกันในระยะเวลาพัฒนาที่จำกัด มักจะทดสอบซอฟต์แวร์เมื่อเสร็จหรือใกล้เสร็จในทีเดียว

ข้อดีของ Big Bang Model

- ใช้งบประมาณไม่สูง เหมาะกับโปรเจกต์ขนาดเล็ก
- ความยืดหยุ่นในการทำงานสูง สามารถทำงานได้อย่างอิสระ
- ไม่ต้องเสียเวลาวางแผนนาน หรือจัดทำเอกสารให้วุ่นวาย

ข้อเสียของ Big Bang Model

- มีความเสี่ยงสูงที่จะพบข้อผิดพลาด หรือบัคจำนวนมาก
- อาจไม่ได้ฟีเจอร์ตามที่ผู้จ้าง requirement มาทุกอย่าง
- ไม่เหมาะกับโปรเจกต์ขนาดใหญ่ หรือที่มีความซับซ้อนสูง